APUNTES. MODULO 2. HERENCIA DE CLASES (relación ES-UN)

Objetivo: entender el concepto de Herencia de Clases

En Programación Orientada a Objetos (POO) las clases pueden depender unas de otras, con una relación ES-UN, también llamada herencia.

La herencia es una forma de reutilizar código: una clase “hija” (subclase) hereda propiedades y métodos de una clase “padre” (clase base).

1) ¿Qué es la herencia?

Hay herencia cuando puedes afirmar que una cosa **ES-UN** tipo de otra.

- Un perro ES un animal.
- Un gato ES un animal.
- Un pájaro ES un animal.

La clase base recoge lo común. Las subclases añaden lo específico.

2) Regla: “ES-UN” vs “TIENE-UN”

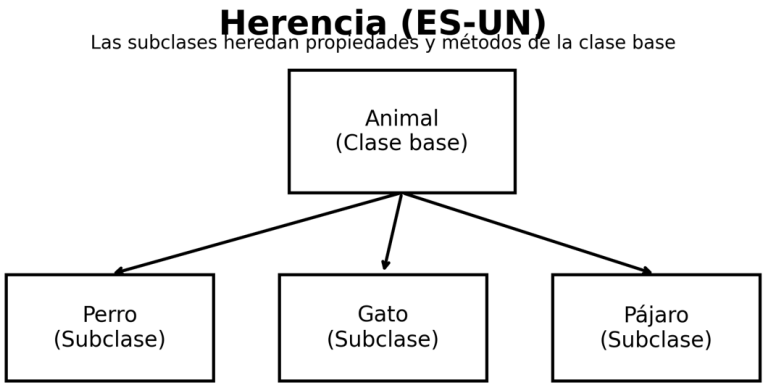
- Si puedes decir “X ES un Y” → herencia.
- Si puedes decir “X TIENE un Y” → composición.

Ejemplos:

- Un perro ES un animal → herencia.
- Un perro TIENE un collar → composición (otra relación distinta).

3) Imagen/diagrama de herencia

En el diagrama, Animal es la clase base y Perro/Gato/Pájaro son subclases:



4) Ejemplo en C#

En C#, la herencia se escribe con dos puntos (:).

Ejemplo: `class Perro : Animal`

4.1) Clase base: Animal

```
public class Animal
{
    public string Nombre { get; set; }

    // Método común (todas las subclases lo tienen)
    public void Dormir()
    {
        Console.WriteLine($"{Nombre} está durmiendo...");
    }
}
```

4.2) Subclases: Perro y Gato

Perro y Gato heredan Nombre y Dormir() sin tener que reescribirlos.

```
public class Perro : Animal
{
    // Método específico del perro
    public void Ladrar()
    {
        Console.WriteLine($"{Nombre}: ¡Guau guau!");
    }
}

public class Gato : Animal
{
    // Método específico del gato
    public void Maullar()
    {
        Console.WriteLine($"{Nombre}: Miau...");
    }
}
```

4.3) Probar en Main()

```
static void Main()
{
    var p = new Perro { Nombre = "Toby" };
    p.Dormir();  // heredado de Animal
    p.Ladrar();  // propio de Perro

    var g = new Gato { Nombre = "Luna" };
    g.Dormir();  // heredado de Animal
    g.Maullar(); // propio de Gato
}
```

5) Un paso más: polimorfismo

Una ventaja enorme de la herencia es que puedes tratar a todas las subclases como su clase base. Eso permite recorrer una lista de animales sin preocuparte del tipo concreto.

5.1) Método virtual y override (opcional, pero recomendable)

Si quieres que cada animal “hable” de forma distinta, define un método

Virtual en la base y override en las subclases.

```
public class Animal
{
    public string Nombre { get; set; }

    public virtual void Hablar()
    {
        Console.WriteLine($"{Nombre}: (sonido genérico)");
    }
}

public class Perro : Animal
{
    public override void Hablar()
    {
        Console.WriteLine($"{Nombre}: ¡Guau!");
    }
}

public class Gato : Animal
{
    public override void Hablar()
    {
        Console.WriteLine($"{Nombre}: ¡Miau!");
    }
}
```

5.2) Lista de Animal (polimorfismo)

```
static void Main()
{
    Animal[] animales =
    {
        new Perro { Nombre = "Toby" },
        new Gato { Nombre = "Luna" }
    };

    foreach (var animal in animales)
        animal.Hablar(); // se ejecuta el override correcto según el objeto real
}
```

6) Errores típicos

- Usar herencia solo para “reutilizar” sin que exista ES-UN: debe tener sentido en el mundo real.
- Crear clases base demasiado grandes: la base debe tener lo común, no todo.
- Confundir herencia con composición: “tiene” no es herencia, es composición.

7) Mini-ejercicios

Indica si sería herencia (ES-UN) o composición (TIENE-UN):

1. Un profesor _____ una persona.
2. Una persona _____ un DNI.
3. Un triángulo _____ una figura.